Class 20: Introduction to Software Engineering for Data Products

Session 20 of 36 – Opening the MLOps & Software Engineering Block

Data-driven Systems Engineering

Agenda

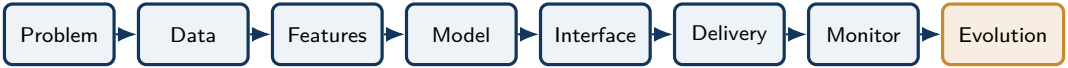
- What is good software?
- Requirements and quality attributes for data products
- Professional software process activities
- Architecture, modularity, and testing
- Software engineering diversity and fundamentals
- Software costs and software products
- Why this matters for ML-enabled systems
- This session opens the MLOps block and now also folds in the legacy “process activities” material as a single unit

Why software engineering matters here

Learning goals

- Reframe ML systems as software systems with data-specific failure modes.
- Understand the four core activities: specification, development, validation, evolution.
- Recognize maintainability as a first-order requirement.
- Distinguish functional requirements from non-functional requirements.
- Connect software architecture and testing choices to ML reliability.

Course Map and Running Cases



Today in the course

Focus: Software quality, architecture, and maintainability.
Bridge: This class bridges from working interfaces to dependable systems by making structure, testing, and evolution explicit.

Fraud case

In the fraud case, architecture separates training logic, saved artifacts, service endpoints, and validation responsibilities.

Pasteurization case

In the pasteurization case, architecture separates simulation, streaming, visualization, and monitoring concerns so the system can evolve safely.

Course thread: From here on, the course treats ML assets as software products with requirements, tests, and operational consequences.

Good software attributes, adapted for ML systems

Attribute	Why it matters
Maintainability	Models, features, and infrastructure all change.
Dependability	Wrong predictions can create financial or physical harm.
Efficiency	Latency and compute cost affect feasibility.
Usability	Operators and analysts must understand outputs.
Traceability	Decisions must be auditable after deployment.

Requirements come before implementation

- Software engineering starts by clarifying what the system must do and under which constraints.
- A strong model does not rescue a system with unclear goals, missing stakeholders, or vague success criteria.
- In data products, requirements must cover both software behavior and data behavior.

Course example

For a fraud service, "predict fraud" is not enough. We also need response time, acceptable error trade-offs, logging, retraining triggers, and who acts on the result.

Functional and non-functional requirements

Functional

- Accept transaction features
- Return a risk score or class
- Stream sensor values
- Display a dashboard

Non-functional

- Respond within a latency budget
- Remain available under load
- Record predictions for audit
- Protect sensitive data

Key point

Many ML system failures are non-functional failures: the model exists, but the service is slow, brittle, opaque, or impossible to maintain.

Software process activities

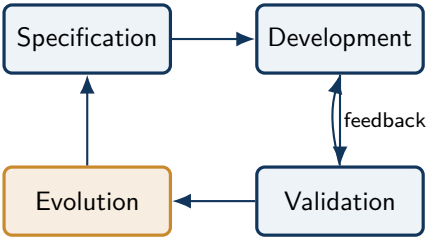
- Specification
- Development
- Validation
- Evolution

- What should the system do?
- How do we build it?
- Did we build the right thing?
- How does it survive change?

ML-specific twist

Evolution is not only about code changes. It also includes new data distributions, updated labels, and model replacement.

Process is iterative, not one-way



- The loop is especially important in ML systems because validation often reveals data problems, not just coding problems.

A better set of software examples

- Customized product: a plant-specific monitoring dashboard for pasteurization.
- Platform component: an internal fraud scoring API reused by several business units.
- System of systems: model service plus feature pipeline plus CRM plus alert routing.
- Data collection system: streaming sensors feeding an online predictor.

How the repo illustrates software evolution

Repo activity

- Notebooks are exploratory artifacts.
- Python modules and Flask services formalize reusable behavior.
- Streamlit apps expose the same logic to stakeholders.
- Tutorials add delivery automation and deployment thinking.

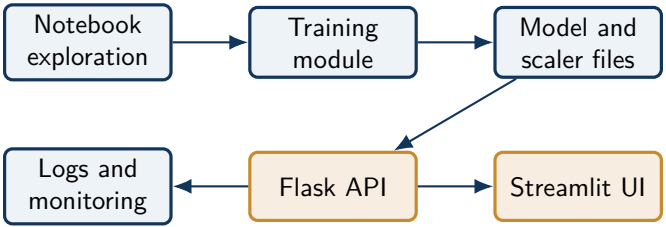
Architecture shapes maintainability

- Architecture is the high-level organization of components and their interactions.
- Good architecture limits coupling, clarifies responsibility, and makes change cheaper.
- Poor architecture turns every model update into a system-wide risk.

Typical decomposition in this course

Training code, saved artifacts, API endpoints, dashboards, and monitoring signals should have clear boundaries even when the project is small.

A simple architecture view of the repo



From notebook logic to reusable software

```
with open("model.pkl", "wb") as f:
    pickle.dump(cls, f)

with open("scaler.pkl", "wb") as f:
    pickle.dump(scaler, f)
```

- In ‘streamlit/modeling.py‘, saving artifacts is the first step from experimentation toward a real software component.
- A model that only lives inside one notebook kernel is difficult to test, reuse, version, or deploy.

Testing is broader than model metrics

- Unit tests check small pieces of logic such as feature functions or request parsing.
- Integration tests check that components work together, for example model artifact plus API endpoint.
- Validation checks whether the delivered behavior satisfies the requirement.
- Regression tests protect against breaking behavior during refactoring or retraining.

Example of behavior worth testing

```python
@app.route("/predict", methods=["POST"])
def predict():
    data = request.get_json(force=True)
    features = np.array(data["features"]).reshape(1, -1)
    pred = model.predict(features)[0]
    return jsonify({"prediction": int(pred)})
```

- A software engineer asks more than “does this run?”
- We also ask what happens when ‘features‘ is missing, malformed, wrongly ordered, or out of expected range.

Service boundary as a software engineering decision

```python
@app.route("/")
def index():
    return jsonify({"message": "Model API running"}), 200

@app.route("/predict", methods=["POST"])
def predict():
    ...
```

- ‘Class6_model_api.py‘ is small, but it already introduces interface contracts, lifecycle management, and API ownership.
- This is where software engineering starts shaping how the ML logic can be consumed by other systems.

Software engineering diversity

- Stand-alone applications
- Interactive transaction-based systems
- Embedded control systems
- Batch processing systems
- Data collection systems
- Systems of systems

Course bridge

The fraud API is a transaction-based system; the pasteurization case is closer to a data collection and monitoring system.

Technical debt in ML systems

- Technical debt appears when short-term convenience creates long-term maintenance cost.
- In ML systems, debt can accumulate in code, pipelines, labels, configuration, and undocumented assumptions.
- Hidden dependencies between data preparation and serving are especially dangerous.
- Fast demos often postpone quality decisions that later become expensive.

Examples in this course

Hard-coded paths, undocumented feature order, and missing validation around API inputs are small examples of debt that can grow quickly.

Software costs and product types

- Software often costs more to maintain than to initially develop.
- Generic products are owned and evolved by the vendor.
- Customized products are scoped and changed with the client.
- In ML systems, monitoring and retraining can push maintenance costs even higher.

Why evolution costs so much

- Requirements change as users learn what they really need.
- Models degrade as data distributions change.
- Interfaces need new validation, observability, and security constraints.
- Every new dependency increases the coordination cost of change.

Practical lesson

The cost of a data product is not the first model notebook. It is the ongoing cost of keeping the whole system trustworthy.

Monitoring system example from the pasteurization service

```
@app.route("/stream")
def stream():
    df, _ = simulate_batch(BATCH_ID)

    def generate():
        for _, row in df.iterrows():
            yield f"data: {json.dumps(row.to_dict())}\\n\\n"
```

- ‘pasteurization/serving.py‘ is a strong example of a data collection and monitoring system rather than a simple batch predictor.
- It helps distinguish software categories using code students can inspect in the repo.

Discussion prompt

Question

If the fraud model is accurate today but the deployment code is brittle, do we have a good system?

- This forces students to separate model quality from system quality.
- It also motivates requirements, testing, deployment, and monitoring lectures.

Papers and materials

Papers and materials

Sommerville, Software Engineering; Chip Huyen, Designing Machine Learning Systems; Martin Fowler, trade-offs in architecture; Nahar et al., The AI lifecycle in production; The New Stack.

From This Class to the Next

What stays with us

- Model quality and software quality are related but not interchangeable.
- Architecture, testability, and maintainability determine whether an ML idea can mature into a dependable system.
- Data-driven applications still obey core software engineering trade-offs.

Project use

For course projects, this is where notebook code should start becoming modular, reviewable, and robust enough to support integration and change.

Next connection

Class 22 moves into requirements engineering, turning stakeholder needs and quality attributes into testable specifications before design and code decisions are judged.

Course thread: Engineering discipline around code and process sets up the same discipline around requirements: knowing precisely what "good" means before we build toward it.